

Práctica 2 Visualización: Gramática de Gráficos.

DataOps

En la práctica desarrollaremos una visualización de la distribución de rentas en Canarias siguiendo los principios de **DataOps** mediante la herramienta **Dagster** y la librería **plotnine**. El conjunto de datos se encuentra en el fichero `distribucion-renta-canarias.csv`. Estos datos están disponibles en:

https://www3.gobiernodecanarias.org/istac/statistical-visualizer/visualizer/collection.html?resourceType=collection&agencyId=ISTAC&resourceId=E30325A_000001

Ficheros disponibles en:

https://drive.google.com/drive/folders/1eZrXpVPPSyNbo4aJ-GgMCG8R_xmBJrB7?usp=sharing

Conceptos Clave en Dagster

Para organizar nuestro flujo de trabajo, utilizaremos los siguientes componentes:

- **Assets (Software-Defined Assets):** El núcleo de Dagster. Un asset es un objeto persistente (DataFrame, CSV, imagen) producido por el pipeline. Se definen con la directiva `@asset` y permiten rastrear el linaje (de dónde viene el dato) y su estado.
- **Ops:** Son las unidades mínimas de computación que ejecutan acciones lógicas (ej. enviar un correo, mover un archivo). *Los Ops se usan para tareas que no producen un objeto de datos concreto.*
- **Asset Checks (Validaciones):** Reglas que verifican la calidad de un asset (ej. "el archivo debe tener 88 municipios"). Si un check falla, Dagster puede detener el pipeline.
- **Jobs:** Agrupaciones de assets que se ejecutan conjuntamente.
- **Schedules y Sensors:** Mecanismos de automatización. Los *Schedules* ejecutan tareas por tiempo (ej. cada lunes), mientras que los *Sensors* reaccionan a eventos externos (ej. la aparición de un nuevo CSV).

Metodología de Trabajo

No ejecutaremos scripts de forma aislada; Dagster orquestará la ejecución. Desde la **interfaz web**, "materializaremos" los assets, lo que permite monitorizar el flujo, consultar metadatos y asegurar que el proceso se detenga si ocurre un error, garantizando la integridad de los datos.

Para cumplir con la **Integración Continua (CI)**, utilizaremos **Git** para el control de versiones, permitiendo que futuros cambios se validen mediante Git Actions.

Pasos para resolver la práctica

1. Preparación del entorno:

- Crear el entorno virtual de Python en la carpeta del proyecto:

```
python -m venv venv
```

- Activarlo: `venv\Scripts\activate`
- Instalar los frameworks con pip :
`dagster`, (pip install dagster)
`dagster-webserver` (pip install dagster-webserver)
`pandas` y `plotnine`.

2. Test de comunicación:

Verificar la instalación materializando el script `test-asset.py` desde la interfaz de Dagster.

```
dagster dev -f ruta de test-assets.py
```

3. Análisis de la Gramática de Gráficos:

Antes de programar, completa la siguiente tabla analítica para tu visualización:

Componente	Descripción / Variable
Dataset	
Geometría	
Estéticas (Aesthetics)	
Escalas	
Etiquetas	

- Prototipado:** Crear `lab-renta.py` para diseñar el gráfico de forma iterativa.
- Integración final:** Convertir el proceso de carga, limpieza, unión y visualización en una cadena de assets interconectados en Dagster.
- Mejorar el gráfico añadiendo los nombres de municipio, para ello será necesario crear un asset que integre el fichero `codislas.csv`

7. Ampliar la visualización con el fichero de nivelesestudios.xlsx